Evaluation Type: Console Application Framework

Details

Weighting: 0%
Submission Instructions: Merge this completed lab into you GitHub main branch
Due Date: Sept. 4, 2026

Overview

In this exercise, you will build a reusable framework for Java console programs and then extend that framework to create a complete application.

The goal is to separate the general structure of a console program from the specific behaviour of one application. Instead of writing the same menu loop over and over, the framework will handle the common program flow for you.

What you will do

- create an abstract ConsoleProgram class;
- define a run() method that controls the overall program flow;
- add abstract methods that subclasses are required to implement;
- add hooks that subclasses may override when custom behaviour is needed;
- add utility methods for reading console input;
- create a new class that extends ConsoleProgram; and
- implement the application-specific menu options and actions.

The key idea is that the framework controls when things happen**,** while the subclass **decides** what those actions actually do**. By the end of** the exercise, you will have a working console application built on top of a reusable Java framework.

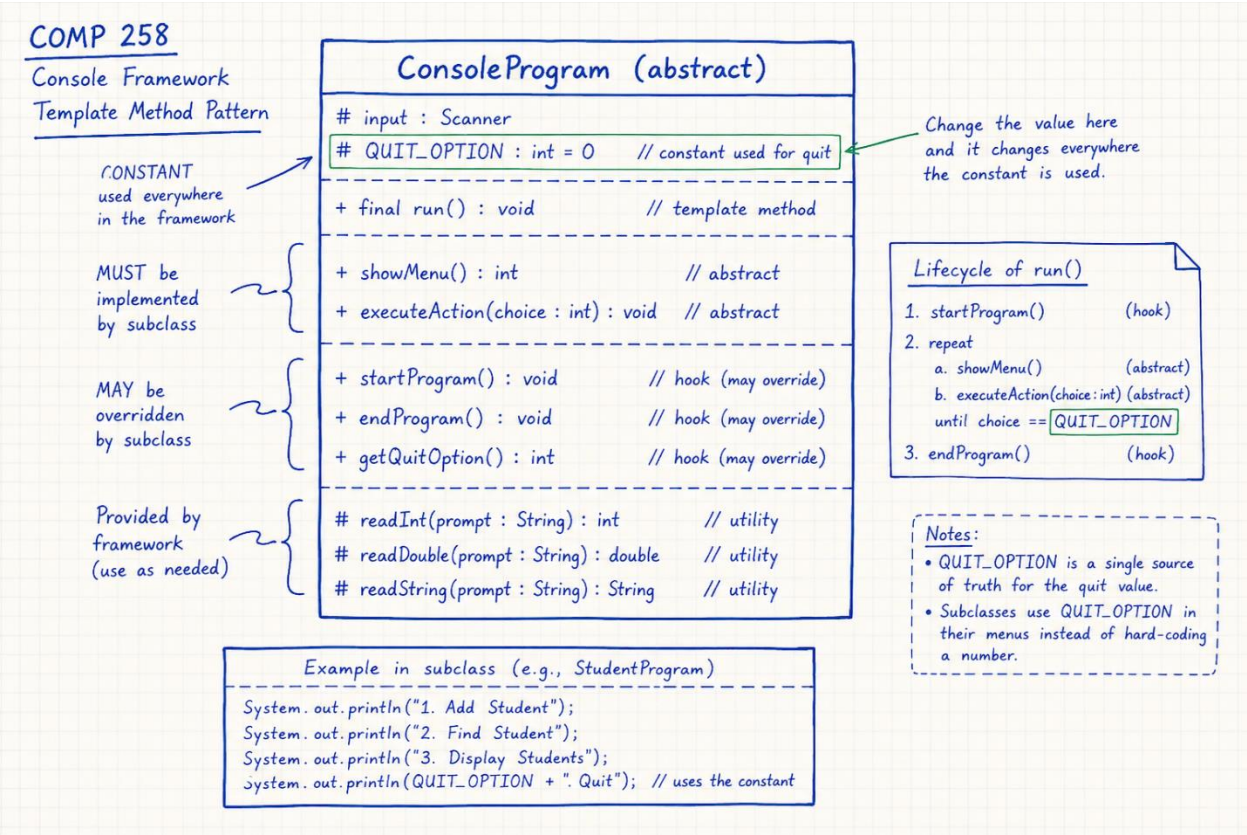
Skills Summary

This exercise will give you practice with:

- abstract classes;
- inheritance;
- method overriding;
- polymorphism;
- hooks;
- reusable framework design; and
- the **Template Method design pattern**.

Instructions

1. *Create the framework:*



2. *Declare the instance variables*

```
protected Scanner input = new Scanner(System.in);
protected static final int QUIT_OPTION = 0;
```

3. *Code the template method:*

```
public final void run() {

    startProgram();
    int choice;
    do {
        choice = showMenu();
```

```
            if (choice != QUIT_OPTION) {
                executeAction(choice);
            }

        } while (choice != QUIT_OPTION);
        endProgram();
    }
```

4. *Provide implementation for the hooks*

```
public void startProgram() {
        System.out.println("Program starting...");
    }

    public void endProgram() {
        System.out.println("Program ended.");
    }
```

5. *Implement Utility Methods*

For example:

```
protected int readInt(String prompt) {

        System.out.print(prompt);

        while (!input.hasNextInt()) {
            System.out.println("Please enter a whole number.");
            input.nextLine();
            System.out.print(prompt);
        }

        int value = input.nextInt();
        input.nextLine();

        return value;
    }
```

6. *Extend the framework into a concrete class*

Create a Java console application that extends the provided ConsoleProgram framework.

Your application will manage an array containing **20 randomly generated integers**. When the program starts, create the array and fill it with random values from **1 to 100**.

The program must provide the following menu options:

1. **Print Numbers**
   Display all values currently stored in the array. When displaying positions to the user, use positions **1 through 20**, even though Java array indexes begin at 0.
2. **Search**
   Ask the user to enter a number to search for. Perform a linear search through the array and report the **position of the first occurrence** of that value. If the number is not found, display an appropriate message.
3. **Find and Replace**
   Ask the user for a value to find and a replacement value. Locate the **first occurrence** of the value and replace it with the new value. Display the human-readable position where the replacement occurred. If the original value is not found, display an appropriate message.
4. **Quit**
   Use the QUIT_OPTION constant inherited from the ConsoleProgram framework rather than hard-coding the quit value.

Your class should extend ConsoleProgram and provide implementations for the required framework methods, including:

```
showMenu()
executeAction(int choice)
```

Use the framework's startProgram() hook to initialize the array and display an appropriate program heading.

Your solution should also include suitable helper methods for the application-specific operations, such as printing the array, searching for a value, and performing the find-and-replace operation.

Requirements

- Store the numbers in an int[].
- The array must contain exactly 20 elements.
- Random values must be between 1 and 10 inclusive.
- Searches must return only the first occurrence.
- Use -1 internally to indicate that a value was not found.

- Array indexes should remain zero-based internally.
- Positions displayed to the user must be one-based.
- Avoid duplicating search logic. The search and find-and-replace operations should share a common search method where appropriate.
- Do not rewrite the framework's run() method. The application should work by extending and customizing the existing framework.

The completed program should demonstrate how a general-purpose framework can be extended with application-specific behaviour while making use of **inheritance, abstract methods, hooks, arrays, loops, method reuse, and linear search**.

